

Guia do Datathon 7MLET – Plataforma de Experimentação Adaptativa (Multi-Armed Bandit)

Base escolhida: [Bank Marketing \(henriqueyamahata\)](#) – bank-additional-full.csv , 41.188 registros, 20 features + target y (assinou depósito a prazo: yes/no).

Divisão de responsabilidades:

Pessoa	Frente	Etapas principais
Gabriel	Organização e Engenharia de Dados	Etapas 0 e 1; Etapa 2 junto com Adryen
Adryen	Modelos	Etapa 2 (com Gabriel) e Etapa 3
Bertelli	Validação e MLOps	Etapas 4 e 7
Matheus	Serviço e Infra	Etapas 5 e 6
Todos	Apresentação	Etapa 8

Gestão do projeto: dependências e ambiente gerenciados com [uv](#) (`pyproject.toml` + `uv.lock`), aceito explicitamente pelo enunciado como alternativa ao `requirements.txt` .

Entendendo o problema antes de codar (leitura obrigatória para os 4)

O desafio pede um **sistema de decisão adaptativa**, não um classificador tradicional. A diferença conceitual é o coração do projeto:

- **Classificador tradicional**: aprende com dados históricos e prevê "este cliente vai converter?". A decisão fica congelada até o próximo retreino.
- **Bandit (Thompson Sampling, Epsilon-Greedy, UCB)**: a cada cliente, o sistema **escolhe uma ação** (qual oferta/canal/abordagem apresentar), **observa a recompensa** (converteu ou não) e **atualiza sua crença** sobre qual ação é melhor. Ele equilibra **exploração** (testar ações incertas) e **exploração** (usar a ação que parece melhor).

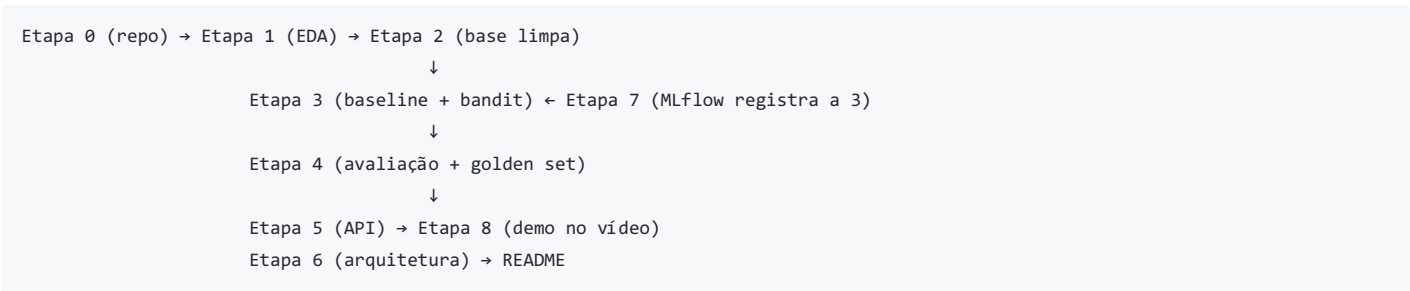
O **problema-chave da nossa base**: o dataset tem *uma* oferta (depósito a prazo) — não existem "várias ofertas" para escolher. Então precisamos **definir o que são os braços (arms)** do bandit. A escolha recomendada (simples e defensável):

Braços = canal/abordagem de contato. Ex.: `contact` (cellular vs telephone) → 2 braços. Se quisermos mais granularidade: `contact × day_of_week` → 10 braços. A recompensa é `y == "yes"` (converteu = 1, não converteu = 0).

Isso é fiel ao enunciado ("qual oferta, mensagem ou **próximo passo** apresentar") e permite simular o bandit com dados reais logados. Documentem essa decisão no README — a banca valoriza a formulação do problema.

Como avaliar um bandit com dados históricos (offline)? Usamos o **método de replay (rejection sampling)**: percorremos o dataset em ordem; para cada cliente, o bandit escolhe um braço; se o braço escolhido coincide com a ação que foi realmente executada no histórico (ex.: o banco de fato ligou por cellular), contabilizamos a recompensa observada e atualizamos o bandit; se não coincide, descartamos o registro. É o padrão da literatura (Li et al., 2011) e é simples de implementar. Expliquem isso no notebook — demonstra maturidade técnica.

Fluxo de dependências entre as etapas:



Etapa 0 – Organização do projeto

Responsável: Gabriel (com input de todos no primeiro dia)

0.1 Criar o repositório

- **O que fazer**: Repositório público no GitHub com nome no padrão `datathon-7mlet-grupo-XX` . Adicionar os 4 como colaboradores.
- **Por quê**: É entregável obrigatório e será o único ponto de entrega — tudo (código, README, notebooks) vive nele.
- **Como fazer**: GitHub → New repository → público → inicializar com README. Definir branch `main` protegida (opcional) e combinar fluxo simples: cada um trabalha em branch própria (`gabriel/eda` , `adryen/bandit` ...) e abre Pull Request. Com 4 pessoas mexendo em notebooks, isso evita conflitos de merge

dolorosos.

- **Conexão:** Todas as etapas seguintes commitam aqui.

0.2 Estrutura de pastas

- **O que fazer:** Criar o esqueleto:

```
datathon-7mlet-grupo-XX/
├── README.md
├── pyproject.toml      # dependências gerenciadas pelo uv
├── uv.lock             # lockfile – commitar sempre
├── data/
│   ├── raw/           # bank-additional-full.csv (ou instrução de download)
│   └── processed/     # base tratada (saída da Etapa 2)
├── notebooks/
│   ├── 01_eda.ipynb
│   └── 02_bandit.ipynb
├── src/
│   ├── data_prep.py   # funções de limpeza (Gabriel)
│   ├── bandit.py      # classes dos algoritmos (Adryen)
│   └── api.py         # FastAPI (Matheus)
├── models/            # artefatos salvos (estado do bandit, encoders)
└── mlruns/            # tracking do MLflow (ou .gitignore se ficar grande)
```

- **Por quê:** Código organizado vale dentro dos 70% de "validação técnica global". Separar `src/` de `notebooks/` permite que a API (Etapa 5) importe as mesmas funções do notebook – sem copiar/colar código.
- **Como fazer:** `mkdir` + arquivos vazios com docstring inicial. Adicionar `.gitignore` (Python padrão + `data/raw/*.csv` se o arquivo for pesado, com instrução de download no README).
- **Conexão:** Gabriel escreve em `src/data_prep.py`, Adryen em `src/bandit.py`, Matheus importa ambos em `src/api.py`.

0.3 Dependências com uv

- **O que fazer:** Inicializar o projeto com `uv` e declarar as dependências no `pyproject.toml`:

```
uv init --name datathon-7mlet
uv add pandas numpy scikit-learn matplotlib seaborn fastapi "uvicorn[standard]" mlflow pydantic
uv add --dev jupyter pytest httpx
```

O `uv` cria/atualiza o `pyproject.toml` e o `uv.lock` automaticamente. **Commitar os dois.**

- **Por quê:** O enunciado aceita `pyproject.toml` como entregável. O `uv.lock` fixa versões exatas de toda a árvore de dependências – qualquer pessoa (incluindo a banca) reproduz o ambiente idêntico com um comando, sem "funciona na minha máquina". Separar dev-dependencies (`jupyter`, `pytest`) mantém claro o que a API precisa em produção vs o que é só desenvolvimento.
- **Como fazer no dia a dia do grupo:**

```
uv sync          # cria .venv e instala tudo do lockfile (rodar após cada git pull)
uv run jupyter lab # roda qualquer comando dentro do ambiente, sem ativar venv
uv run pytest -q
uv run uvicorn src.api:app --reload
uv add <pacote>   # quem adicionar lib nova commita pyproject.toml + uv.lock no mesmo PR
```

- **Conexão:** Pré-requisito de execução de todas as etapas; as instruções de execução do README (Etapas 5 e 6) usam `uv sync` + `uv run`.

0.4 README inicial (esqueleto)

- **O que fazer:** README com seções vazias que serão preenchidas ao longo do projeto: Problema, Base de dados (link Kaggle), Como executar, Formulação do bandit, Resultados, Golden Set, Arquitetura em nuvem, MLOps, Limitações.
- **Por quê:** O enunciado exige que **toda a documentação** esteja consolidada no README. Criar o esqueleto no dia 1 evita correria no final e deixa claro quem preenche o quê.
- **Como fazer:** Cada seção ganha um comentário `<!-- responsável: Fulano -->`.
- **Conexão:** Gabriel preenche "Base de dados", Adryen "Formulação do bandit" e "Resultados", Bertelli "Golden Set" e "MLOps", Matheus "Como executar a API" e "Arquitetura".

Etapa 1 — Base Kaggle e EDA

Responsável: Gabriel

1.1 Carga e sanidade dos dados

- **O que fazer:** Ler o CSV corretamente e validar o shape esperado (41.188×21).
- **Por quê:** O arquivo usa **separador** `;` (não vírgula) e tem quebras de linha `\r\n`. Errar isso corrompe tudo rio abaixo.
- **Como fazer:**

```
import pandas as pd
df = pd.read_csv("data/raw/bank-additional-full.csv", sep=";")
assert df.shape == (41188, 21)
df.info()
```

Verificar tipos: 10 numéricas, 10 categóricas + target. Checar duplicatas (`df.duplicated().sum()` — existem ~12 no dataset, documentar e remover).

- **Conexão:** É a porta de entrada do pipeline. O notebook `01_edu.ipynb` é entregável direto da Etapa 1.

1.2 Análise do target e desbalanceamento

- **O que fazer:** Calcular a taxa de conversão global (`y == "yes" ≈ 11,3%`) e visualizar.
- **Por quê:** O desbalanceamento (~89/11) muda tudo: acurácia é métrica inútil aqui, e a taxa base de 11,3% é exatamente o **baseline natural** que o bandit precisa superar. Esse número aparecerá de novo nas Etapas 3 e 4.
- **Como fazer:** `df["y"].value_counts(normalize=True)` + gráfico de barras. Escrever uma célula markdown explicando a implicação.
- **Conexão:** Alimenta a escolha de métricas do Bertelli (Etapa 4) e o baseline do Adryen (Etapa 3).

1.3 Análise por braço (a parte mais importante da EDA)

- **O que fazer:** Cruzar a conversão com as variáveis candidatas a braço: `contact`, `day_of_week`, `month`. Montar tabelas de taxa de conversão por grupo.
- **Por quê:** É aqui que vocês **justificam a escolha dos braços**. Spoiler do dataset: ~~cellular converte 14,7% vs telephone ~5,2% — uma diferença enorme, o que torna o problema de bandit interessante (existe um braço claramente melhor a ser descoberto).~~ `day_of_week` tem variação menor (10,8% a ~12,1%), útil se quiserem braços compostos.
- **Como fazer:**

```
df.groupby("contact")["y"].apply(lambda s: (s == "yes").mean())
pd.crosstab(df["contact"], df["day_of_week"],
            values=(df["y"] == "yes"), aggfunc="mean")
```

Visualizar com heatmap. Concluir em markdown: "escolhemos braços = X porque...".

- **Conexão:** Decisão estrutural que define a Etapa 2 (como a base é preparada) e a Etapa 3 (quantos braços o bandit tem).

1.4 Vazamento temporal e variáveis problemáticas

- **O que fazer:** Documentar e marcar para descarte a coluna `duration`. Analisar também `pdays` (999 = nunca contatado, em ~96% dos casos) e os "unknown" das categóricas.
- **Por quê:** O próprio enunciado exige descartar colunas de vazamento (`duration` só é conhecida **depois** da ligação — usá-la é trapaça e a banca vai procurar exatamente isso). `pdays=999` é um código sentinela, não um número real — tratá-lo como numérico distorce qualquer modelo.
- **Como fazer:** Célula markdown explicando o porquê do descarte de `duration` (citar a nota do próprio dataset). Para `pdays`, criar flag binária `was_contacted_before = (pdays != 999)`. Para "unknown", contabilizar por coluna e decidir: manter como categoria própria (recomendado — é informação: "cliente que não informou") em vez de imputar.
- **Conexão:** Regras que Gabriel implementa como código na Etapa 2.

1.5 Contexto socioeconômico e distribuições

- **O que fazer:** Explorar as 5 variáveis macro (`emp.var.rate`, `cons.price.idx`, `cons.conf.idx`, `euribor3m`, `nr.employed`), correlações entre elas e distribuições de `age`, `campaign`.
- **Por quê:** As macros são altamente correlacionadas entre si (`euribor3m × emp.var.rate ~0,97`) — se o grupo evoluir para um bandit **contextual**, saber disso evita redundância de features. Também mostra domínio do dado à banca.
- **Como fazer:** `df[macro_cols].corr()` + heatmap; histogramas das numéricas; boxplot de `age` por `y`.
- **Conexão:** Se Adryen implementar variante contextual na Etapa 3, usa este mapa de features.

Etapa 2 — Preparação da Base

Responsáveis: **Gabriel e Adryen** (Gabriel lidera o pipeline de limpeza; Adryen valida braços, features e o contrato de dados que o bandit vai consumir)

2.1 Pipeline de limpeza como código reutilizável

- **O que fazer:** Transformar as decisões da EDA em funções puras em `src/data_prep.py`:

```
def load_raw(path) -> pd.DataFrame: ...
def clean(df) -> pd.DataFrame:
    # remove duplicatas, dropa duration, cria was_contacted_before,
    # mantém 'unknown' como categoria
def build_bandit_frame(df) -> pd.DataFrame:
    # colunas: arm (ex.: contact), reward (y=='yes' -> int),
    # + features de contexto
```

- **Por quê:** A Etapa 3 (simulação), a Etapa 4 (golden set) e a Etapa 5 (API) precisam aplicar **exatamente a mesma transformação**. Se a limpeza viver só dentro do notebook, a API do Matheus vai divergir do modelo do Adryen — bug clássico de treino/serving skew.
- **Como fazer:** Funções pequenas, com docstring, sem estado global. O notebook `02_bandit.ipynb` importa de `src/` (`from src.data_prep import clean`). Salvar a saída em `data/processed/bandit_frame.parquet`.

- **Conexão:** É o contrato de dados do projeto: Adryen consome `arm + reward` (+ contexto), Matheus consome as mesmas features na API.

2.2 Encoding das features de contexto

- **O que fazer:** Definir e persistir o encoding das categóricas (one-hot via `pd.get_dummies` ou `OneHotEncoder` do sklearn) e a lista final de colunas.
- **Por quê:** O bandit "puro" (não contextual) nem usa as features — mas o **golden set** (Etapa 4) e uma eventual variante contextual precisam. Persistir o encoder (`joblib.dump`) garante que a API transforme um cliente novo do mesmo jeito.
- **Como fazer:** `ColumnTransformer` com `OneHotEncoder(handle_unknown="ignore")` para categóricas + `passthrough/scaler` nas numéricas. Salvar em `models/preprocessor.joblib`.
- **Conexão:** Matheus carrega este artefato na Etapa 5; Bertelli loga o artefato no MLflow na Etapa 7.

2.3 Ordenação temporal e split

- **O que fazer:** Manter o dataset na ordem original (ele já vem ordenado por data, maio/2008 → nov/2010) para a simulação de replay. Definir também um split simples (ex.: primeiros 80% para "simulação/aprendizado", últimos 20% para o golden set e checagens).
- **Por quê:** Bandit é um algoritmo **online** — a ordem dos eventos importa. Embaralhar destruiria a narrativa temporal (e o dataset tem forte drift: a crise de 2008-2010 muda as taxas de conversão ao longo do tempo, o que é ótimo argumento para "regras fixas envelhecem, bandits se adaptam" no pitch).
- **Como fazer:** Não fazer `shuffle`. Documentar no notebook que a ordem = ordem cronológica.
- **Conexão:** A simulação do Adryen (3.3) percorre esse frame em ordem; o argumento do drift entra no vídeo (Etapa 8).

Etapa 3 — Baseline e estratégia algorítmica

Responsável: Adryen

3.1 Baseline determinístico

- **O que fazer:** Implementar duas políticas de controle e medir a taxa de conversão de cada:
 1. **Regra fixa ingênua:** sempre escolher o mesmo braço (ex.: sempre `telephone`) → conversão ~5,2%.
 2. **Melhor braço histórico (oráculo estático):** sempre `cellular` → ~14,7%.
- **Por quê:** O enunciado exige métrica comparativa clara. Ter **dois** baselines conta uma história melhor: o bandit deve esmagar o (1) e **convergir para perto do (2)** — mostrando que ele *descobre sozinho* o melhor braço sem conhecimento prévio, que é o valor de negócio.
- **Como fazer:** Sobre o frame da Etapa 2, filtrar por braço e calcular a média de `reward`. Guardar os números em variáveis para o gráfico comparativo (3.4).
- **Conexão:** Números registrados no MLflow (Etapa 7) e citados no README e no vídeo.

3.2 Implementar os algoritmos em `src/bandit.py`

- **O que fazer:** Classes com interface comum:

```
class EpsilonGreedy:
    def __init__(self, n_arms, epsilon=0.1): ...
    def select_arm(self) -> int: ...
    def update(self, arm, reward): ...

class ThompsonSampling:
    def __init__(self, n_arms, alpha_prior=1, beta_prior=1): ...
    # cada braço tem uma Beta(alpha, beta);
    # select_arm: amostra de cada Beta, escolhe o argmax
    # update: reward=1 -> alpha+=1 ; reward=0 -> beta+=1
```

- **Por quê:** Recompensa binária (converteu/não) casa perfeitamente com o modelo **Beta-Bernoulli** do Thompson Sampling — é o exemplo canônico da literatura, fácil de explicar para a banca ("cada braço tem uma distribuição de crença; amostramos delas e escolhemos o maior sorteio; braços incertos ainda ganham chances = exploração natural"). O enunciado pede **priors documentados**: `Beta(1,1)` é a prior uniforme (não sabemos nada), documentem isso explicitamente.
- **Como fazer:** ~30 linhas por classe, só `numpy`. Escrever um teste rápido de sanidade: bandit com braços de probabilidade conhecida (0.05 vs 0.15 simulados) deve convergir para o melhor. Isso vira célula do notebook e prova que a implementação está certa antes de tocar nos dados reais.
- **Conexão:** A interface `select_arm / update` é a mesma que a API do Matheus usará (Etapa 5) para o endpoint de decisão e o de feedback.

3.3 Simulação por replay nos dados reais

- **O que fazer:** Percorrer o frame ordenado; a cada linha, o bandit escolhe um braço; se coincidir com o braço logado (`contact` real daquela linha), computar a recompensa e atualizar; senão, pular a linha.
- **Por quê:** É a forma estatisticamente honesta de avaliar uma política de decisão com dados observacionais — só sabemos o resultado da ação que **de fato** aconteceu. Explicar isso no notebook rende pontos de maturidade técnica.
- **Como fazer:**

```

matched, rewards = 0, []
for _, row in frame.iterrows():
    arm = bandit.select_arm()
    if arm == row["arm"]:
        bandit.update(arm, row["reward"])
        rewards.append(row["reward"])
        matched += 1
conversao_bandit = np.mean(rewards)

```

Rodar com múltiplas seeds (ex.: 10) e reportar média $\pm$ desvio — bandits são estocásticos e a banca pode perguntar sobre variância. Guardar também a **conversão acumulada ao longo do tempo** e a **fração de escolhas por braço ao longo do tempo**.

- **Conexão:** Cada rodada (algoritmo $\times$ hiperparâmetro $\times$ seed) vira um *run* no MLflow do Bertelli (Etapa 7).

3.4 Análise exploração $\times$ exploração e gráfico final

- **O que fazer:** Produzir os gráficos-chave: (a) conversão acumulada — bandit vs baseline fixo vs melhor braço; (b) % de vezes que cada braço foi escolhido ao longo das rodadas; (c) para Thompson, evolução das distribuições Beta (plotar as densidades no início, meio e fim).
- **Por quê:** O enunciado pede explicitamente "análise de exploração" e "análise do trade-off entre exploração e conversão". O gráfico (b) conta a história visualmente: no começo o bandit explora (~50/50), depois converge para o braço bom. É a imagem principal do vídeo.
- **Como fazer:** Matplotlib no notebook. Comparar Epsilon-Greedy com 2-3 valores de epsilon (0.05, 0.1, 0.3) vs Thompson — mostrar que epsilon alto = mais exploração = conversão menor no longo prazo, e que Thompson reduz exploração automaticamente.
- **Conexão:** Figuras salvas em `reports/figures/` → usadas no README (Adryen preenche "Resultados") e no vídeo (Etapa 8).

Etapa 4 — Avaliação e Casos de Teste

Responsável: Bertelli (números vindos do Adryen)

4.1 Consolidar métricas de avaliação

- **O que fazer:** Montar a tabela final de métricas:

Política	Conversão (replay)	Regret acumulado	N amostras usadas
Regra fixa (telephone)	~5,2%	alto	—
Melhor braço histórico	~14,7%	0 (oráculo)	—
Epsilon-Greedy ($\epsilon=0.1$)	preencher	preencher	preencher
Thompson Sampling	preencher	preencher	preencher

- **Por quê:** "Modelo funcionando e superando o baseline" é o critério de maior peso (dentro dos 70%). Uma tabela única e limpa é o que a banca vai procurar primeiro. **Regret** (diferença acumulada entre a recompensa do oráculo e a da política) é a métrica clássica de bandits e mostra vocabulário correto.
- **Como fazer:** `regret_t = t * p_melhor_braço - sum(rewards_até_t)` ; plotar regret acumulado por política (curva que "achata" = política aprendeu). Colocar tabela no README e no notebook.
- **Conexão:** Estes são os números que o próprio Bertelli registra no MLflow (Etapa 7) e que abrem o vídeo.

4.2 Golden Set — 5 clientes de teste

- **O que fazer:** Escolher 5 clientes representativos e mostrar, para cada um, a recomendação do sistema e uma justificativa em uma frase. Sugestão de perfis:
 1. Jovem estudante, contato prévio com sucesso (`outcome=success`) — perfil de alta propensão.
 2. Aposentado, nunca contatado — propensão média-alta (aposentados convertem bem neste dataset).
 3. Blue-collar, casado, com empréstimos — perfil de baixa propensão histórica.
 4. Cliente já contatado 6+ vezes nesta campanha (`campaign` alto) — caso de saturação.
 5. Cliente com muitos `unknown` — caso de dado faltante, testa robustez.
- **O que mostrar:** Para o bandit puro, a "recomendação" é o braço escolhido (canal) + as probabilidades estimadas de cada braço (médias das Betas) naquele momento. Se houver variante contextual/modelo de propensão, mostrar também o score do cliente.
- **Por quê:** Entregável explícito da Etapa 4. Além disso, é o **teste de regressão** de vocês: se alguém mudar o pipeline e o golden set mudar de forma absurda, algo quebrou.
- **Como fazer:** Selecionar as 5 linhas do split de teste (últimos 20%), passar pelo `preprocessor` e pelo bandit treinado, montar tabela no README: *cliente* → *features resumidas* → *recomendação* → *faz sentido?* (*sim/não + porquê*).
- **Conexão:** Estes mesmos 5 clientes viram os exemplos de request da API (Etapa 5) e a demonstração ao vivo do vídeo (Etapa 8).

4.3 Testes automatizados mínimos (diferencial barato)

- **O que fazer:** 4-6 testes com `pytest` em `tests/`: (a) `clean()` remove `duration`; (b) shape esperado pós-limpeza; (c) Thompson converge no cenário sintético; (d) API responde 200 com payload do golden set; (e) API rejeita payload inválido (422).
- **Por quê:** "Código organizado" e "casos de teste" estão nos critérios; `pytest -q` verde na tela do vídeo é um diferencial de 10 minutos de esforço.
- **Como fazer:** `pytest` e `httpx` já entram como dev-dependencies na Etapa 0 (`uv add --dev pytest httpx`); rodar com `uv run pytest -q`. Para a API usar `fastapi.testclient.TestClient`.
- **Conexão:** Roda no ambiente local; se sobrar tempo, vira um GitHub Action de 5 linhas (bonus).

Etapa 5 — Serviço demonstrável (API)

Responsável: Matheus

5.1 Desenhar o contrato da API

- **O que fazer:** Definir os endpoints em FastAPI (`src/api.py`):
 - `POST /recommend` — recebe JSON com as features do cliente, retorna `{"arm": "cellular", "arm_probs": {"cellular": 0.14, "telephone": 0.05}, "policy": "thompson"}`.
 - `POST /feedback` — recebe `{"arm": ..., "reward": 0|1}` e chama `bandit.update()`. **Este endpoint é o que torna o sistema "adaptativo" de verdade** — é o loop de aprendizado online.
 - `GET /health` — status + versão do modelo.
- **Por quê:** O enunciado aceita script ou notebook, mas a API com endpoint de feedback demonstra o conceito central do desafio (aprender com respostas observadas) e rende a melhor demo no vídeo. O `/feedback` é o argumento matador: "diferente de um modelo batch, nosso serviço aprende a cada resposta".
- **Como fazer:** Modelos `pydantic` para request/response (validação grátis + documentação automática em `/docs`). Campos com os mesmos nomes/categorias do dataset.
- **Conexão:** Consome `models/preprocessor.joblib` (Gabriel) e a classe de `src/bandit.py` (Adryen) com o estado salvo pós-simulação.

5.2 Persistir e carregar o estado do bandit

- **O que fazer:** Ao final da simulação (Etapa 3), salvar o estado do bandit (os vetores alpha/beta do Thompson) em `models/bandit_state.json`. A API carrega no startup e salva a cada feedback (ou a cada N).
- **Por quê:** Sem isso, a API "esquece" tudo a cada restart. Persistência de estado é o detalhe que separa uma demo de brinquedo de um serviço com cara de produção.
- **Como fazer:** `json.dump({"alpha": [...], "beta": [...]});` no FastAPI, usar evento de `lifespan /startup` para carregar.
- **Conexão:** Este artefato também é logado no MLflow (Etapa 7) como o "modelo" versionado.

5.3 Rodar e documentar a execução

- **O que fazer:** Instruções no README: `uv sync` seguido de `uv run uvicorn src.api:app --reload` + exemplos de `curl` com os 5 clientes do golden set + screenshot do Swagger (`/docs`).
- **Por quê:** Checklist exige "código executável que retorna a predição funcionando perfeitamente" e "instruções claras de execução local". A banca precisa reproduzir em minutos — com `uv`, são exatamente dois comandos.
- **Como fazer:** Testar do zero num ambiente limpo (clonar o repo em outra pasta, rodar `uv sync`, seguir o próprio README). Se travar em algum passo, o README está incompleto.
- **Conexão:** É exatamente o roteiro da demo do vídeo (Etapa 8).

Etapa 6 — Arquitetura-alvo em Nuvem

Responsável: Matheus

6.1 Escrever os parágrafos de arquitetura no README

- **O que fazer:** 1-2 parágrafos descrevendo como o sistema iria ao ar. Sugestão de narrativa em AWS (troque pelo provedor que o grupo domina):

*Os dados brutos e processados ficariam em um bucket **S3** (data lake), com o pipeline de preparação rodando como job agendado (**Lambda** ou **ECS/Fargate** para volumes maiores). A API FastAPI seria empacotada em container **Docker**, publicada no ECR e servida via **ECS Fargate** atrás de um **Application Load Balancer** (alternativa serverless: API Gateway + Lambda). O estado do bandit (parâmetros Beta por braço) viveria no **DynamoDB**, permitindo atualizações de baixa latência a cada feedback e sobrevivência a restarts. O tracking de experimentos usaria **MLflow** com backend em **S3/RDS**. Observabilidade via **CloudWatch** (latência, taxa de erro, distribuição de escolhas por braço e taxa de conversão em janela móvel — este último é o monitor de drift do negócio).*

- **Por quê:** Entregável explícito; diagrama é opcional, mas o texto precisa mostrar que vocês sabem **onde cada peça do que construíram viveria** — mapeamento 1:1 entre o repo e os serviços.
- **Como fazer:** Escrever conectando cada serviço a um artefato real do projeto ("o `bandit_state.json` viraria itens no DynamoDB"). Se sobrar tempo, um diagrama simples no draw.io/Mermaid soma pontos.
- **Conexão:** Mapeia diretamente os artefatos das Etapas 2, 3, 5 e 7 para serviços gerenciados; vira 20 segundos do vídeo.

Etapa 7 — Ciclo de vida MLOps (MLflow)

Responsável: Bertelli (instrumentando o notebook do Adryen)

7.1 Instrumentar a Etapa 3 com MLflow

- **O que fazer:** Envolver cada rodada de simulação em um run do MLflow:

```
import mlflow
mlflow.set_experiment("datathon-bandit")
with mlflow.start_run(run_name="thompson_seed42"):
    mlflow.log_params({"policy": "thompson", "n_arms": 2,
                      "alpha_prior": 1, "beta_prior": 1, "seed": 42})
    # ... simulação ...
    mlflow.log_metrics({"conversao_replay": conv,
                       "regret_final": regret,
                       "n_matched": matched})
    mlflow.log_artifact("models/bandit_state.json")
    mlflow.log_artifact("reports/figures/conversao_acumulada.png")
```

- **Por quê:** Entregável explícito ("registrar os parâmetros do modelo e as métricas da Etapa 3"). Também resolve um problema real do grupo: com 2 algoritmos × vários hiperparâmetros × 10 seeds, sem tracking vocês se perdem nos números.
- **Como fazer:** MLflow local (mlflow ui → <http://localhost:5000>). Logar **priors** (exigência explícita do enunciado), epsilon, seed, e as métricas da tabela da Etapa 4. Um screenshot da UI comparando runs entra no README.
- **Conexão:** Consome a simulação do Adryen (3.3) e as métricas da Etapa 4.1; o screenshot da UI aparece no vídeo como prova do "uso básico de MLflow".

7.2 Parágrafo de ciclo de vida no README

- **O que fazer:** Descrever em poucas linhas o ciclo completo: dados versionados → experimento rastreado → modelo (estado do bandit) como artefato → serving via API → feedback realimenta o modelo → monitoramento de conversão por braço → retreino/reset quando houver drift.
- **Por quê:** Fecha a narrativa de "ciclo de ponta a ponta" que os critérios pedem, e conecta MLflow (offline) com o /feedback (online) — a ponte entre as Etapas 7 e 5 que a maioria dos grupos não explicita.
- **Conexão:** Complementa a Etapa 6 no README.

Etapa 8 — Apresentação Final (Demo Day)

Responsáveis: todos (sugestão de condução abaixo)

8.1 Roteiro do vídeo (5 minutos)

Tempo	Conteúdo	Quem conduz
0:00–0:45	Problema de negócio: regras fixas e A/B tests desperdiçam tráfego; bandit aprende em tempo real. Citar o drift 2008-2010 do dataset como evidência de que regras envelhecem.	Gabriel
0:45–1:45	Dados e formulação: base Kaggle, braços = canal de contato, recompensa = conversão, por que descartamos <code>duration</code> .	Gabriel
1:45–3:00	Modelo e resultados: Thompson vs Epsilon vs baselines; gráfico de conversão acumulada e de escolha de braços; tabela final.	Adryen
3:00–4:15	Demo ao vivo: <code>uvicorn</code> rodando, <code>POST /recommend</code> com 2 clientes do golden set, <code>POST /feedback</code> mostrando o estado do bandit mudando + arquitetura em 3 frases.	Matheus
4:15–5:00	Operação: print do MLflow, golden set validado, limitações (avaliação offline por replay ≠ produção; dados de 2008-2010).	Bertelli

- **Por quê:** Critério de negócio = 30% da nota, e é onde grupos técnicos perdem ponto. Começar pelo problema (não pelo algoritmo) e terminar com limitações passa maturidade.
- **Como fazer:** Gravar a demo **antes** (screencast) e narrar por cima, ou ensaiar 2x com cronômetro — 5 minutos passam voando. OBS Studio ou gravação do Meet/Zoom resolvem. Falar sempre "conversão", "tráfego desperdiçado", "aprendizado em tempo real" — vocabulário de negócio.
- **Conexão:** Cada bloco do vídeo corresponde a uma etapa entregue; nada novo é criado aqui, só contado.

8.2 Ensaio de reprodução (todos, véspera da entrega)

- **O que fazer:** Uma pessoa que **não** escreveu o código (revezar) clona o repo do zero e segue o README à risca: `uv sync`, rodar notebook, subir API com `uv run`, disparar os curls, abrir MLflow.
- **Por quê:** "Sucesso na demonstração prática" está nos critérios. O modo mais comum de perder ponto é o clássico "na minha máquina funciona".
- **Como fazer:** Checklist do enunciado impresso; marcar item por item. O que falhar volta como issue para o dono da etapa.

Cronograma sugerido (ajustem ao prazo real)

Semana	Gabriel	Adryen	Bertelli	Matheus

Semana 1	Repo, estrutura, uv/pyproject (Etapa 0) + início da EDA	Estudo dos algoritmos + implementação sintética (3.2)	Definição das métricas e dos perfis do golden set (4.1, 4.2)	Desenho do contrato da API (5.1) + rascunho da arquitetura (Etapa 6)
2	EDA completa (Etapa 1) + data_prep.py (Etapa 2, com Adryen)	Etapa 2 com Gabriel (braços, encoding, contrato de dados) + simulação replay (3.3)	MLflow instrumentado (7.1) + testes (4.3)	Primeira versão da API (5.1, 5.2)
3	Revisão README (dados)	Análises de exploração (3.4) + tabela de resultados (com Bertelli, 4.1)	Golden set final (4.2) + ciclo MLOps no README (7.2)	API completa, estado persistido e execução no README (5.2, 5.3) + arquitetura (Etapa 6)
4	— Vídeo, ensaio de reprodução e checklist final (todos) —			

Armadilhas para não cair (resumo dos pontos que a banca vai caçar)

1. **duration no modelo** → desclassificação moral imediata; o enunciado avisa duas vezes.
2. **Embaralhar os dados antes do replay** → quebra a lógica online do bandit.
3. **Bandit sem baseline comparável** → o critério pede explicitamente a comparação.
4. **Priors não documentados** no Thompson → exigência textual do enunciado.
5. **README incompleto** → é o único documento; tudo vive nele.
6. **Um único seed** → resultados de bandit variam; reportem média ± desvio.
7. **API que não roda em máquina limpa** → ensaio de reprodução (8.2) existe para isso.